

Report

Name : Wenhao Cao

Student ID: 3750593

Introduction

This is the specification of a quizlet-like mobile app named “Quizzeit.” “Quizzeit” is designed to help students enhance their understanding of high school and university-level Formal Logic concepts including concepts of truth table, deMorgan law, and conditional rule. The app provides a platform where users can test their knowledge on various logic topics through interactive quizzes. By offering a range of customizable quizzes and immediate feedback, "Quizzeit" aims to make learning formal logic engaging and effective.

License:

MIT License

Logical App Overview

App Router: The code defines routing for the app using the “GoRouter” package, enabling efficient navigation across various screens. Here is the detailed analysis:

1. **Initial Route:** The app starts at /login by default, making the login page the landing screen.
2. **Login and Signup Routes:** Simple routes for /login and /signup are defined using GoRoute with no transitions (NoTransitionPage), offering a smooth experience.
3. **Branching and Dynamic Routes:** The quiz section includes routes like /quiz_list for the list and /quiz_list/quiz/:quizId for individual quizzes. The quizId is passed dynamically, while /quiz_result uses extra data for passing quiz results.
4. **StatefulShellRoute:** The StatefulShellRoute.indexedStack manages different app screens (admin, profile, quiz, guide) using a ScaffoldWithNestedNavigation widget, which likely handles the layout and tab navigation through a BottomNavigationBar. This enables smooth switching between different tabs while preserving the state of each section.

State Management: There are two providers, quizProvider and userProvider, using Riverpod for state management in a Flutter app. Here is the detailed analysis:

UserProvider:

1. **State Management:** The UserState holds the user's profile information, including name, username, email, and quiz scores. UserNotifier manages the user state and provides functionality for authentication and profile updates.
2. **User Authentication:** The login and signup methods interact with backend APIs to authenticate the user or create a new account. On successful login, the user state is updated with data from the backend.

3. **Profile Management:** Methods like `updateUser` allow partial updates to user information, while `addWebsiteUrl` and `removeWebsiteUrl` manage the list of URLs linked to the user's profile.
4. **`_syncWithBackend()`:** The method sends the current user state to the backend server via a POST request, ensuring that any local changes are synchronized with the backend. Every time the user's state is updated (e.g., through methods like `updateUser()`, `setQuizScore()`, or modifying the profile), the `_syncWithBackend()` method is triggered to synchronize the changes with the backend server. This ensures that all local updates are consistently reflected on the server.

QuizProvider:

1. **State Management:** The `QuizState` class holds the list of quiz questions. The `QuizNotifier` extends `StateNotifier<QuizState>`, providing methods to fetch quiz data from a backend.
2. **Quiz Fetching:** The `fetchQuiz` method handles HTTP GET requests, retrieving quiz data from different endpoints and parsing the response into a list of `Question` objects.
3. **Quiz Generation:** Various methods like `generateConditionalQuiz` and `generateHardQuiz` call specific API endpoints to generate quizzes of different types or difficulty levels.
4. **Dynamic Data:** The `Question` class constructs a quiz question object from JSON data, supporting both old and new API formats. It shuffles the options to randomize them for each quiz.

Login Page and Signup Page:

1. **Input Fields:** The screen uses `TextEditingController` for capturing email and password inputs, which are then used for login.
2. **Login Process:** Upon pressing the login button, the app attempts to log in via `userNotifier.login()`. If unsuccessful, it will display an error message.
3. **Navigation on Success:** The app listens to `userProvider` with `ref.listen<UserState>`. When the user's username is updated (indicating a successful login), it triggers navigation to the profile page using `context.go('/profile')`.
4. **Error Handling:** An error dialog provides immediate feedback if login fails, enhancing user experience.
5. **SignupScreen:** The screen allows users to create an account by entering their details and verifying passwords, while handling errors with validation and providing navigation to the login page after successful signup.

Quiz Logic:

1. **Quiz Listing:** The `QuizListPage` serves as the main hub where users can browse and select quizzes to attempt. It divides quizzes into two categories: "Random Generated Quizzes" and "Fixed Quizzes."
2. **UI Components:** The page is structured as a scrollable list (`ListView`) displaying various quiz types. Each quiz type (such as "Conditional Quiz", "De Morgan's Quiz", etc.) is represented by a `ListTile` that shows the title of the quiz and the user's score (if

available). A start button on each tile prompts a dialog that confirms the user's intention to start the quiz.

3. **Quiz Selection and Navigation:** Upon confirmation, the dialog triggers the `_startQuiz()` method. This method calls the appropriate quiz generation function (such as `generateConditionalQuiz()`, `generateEasyQuiz()`, etc.) depending on the quiz type. After the quiz is initialized, the app navigates to the quiz-taking page using the quiz's ID (e.g., `/quiz_list/quiz/1` for the Conditional Quiz).
4. **Quiz Initialization:** When the page loads, it triggers the generation of the specific quiz using the quiz ID (e.g., `generateDeMorganQuiz()` for De Morgan's Quiz). An empty list (`_selectedOptions`) is initialized to track the user's answers.
5. **Timer:** A 10-minute countdown timer is displayed, which decreases by one second. If time runs out, the quiz is automatically submitted using the `_onSubmit()` method.
6. **Question Navigation:** Users can navigate between questions with "Next" and "Previous" buttons. `GridView.builder()` renders the answer options, and the selected answers are stored in `_selectedOptions`. The "Question Status" section shows which questions have been answered.
7. **Quiz Submission:** Upon submission, the app calculates the score, updates the user's state using `setQuizScore()`, and navigates to the result page, passing the selected and correct answers.
8. **Quiz Summary:** The `ListView.builder()` loops through the questions and presents each one in a Card. It highlights whether the user's answer was correct or incorrect, using colors to distinguish the two (green for correct, red for incorrect). and after reviewing their results, users can return to the QuizListPage by clicking the "Back to Quiz List" button, allowing them to either retry quizzes or attempt new ones.

Profile Page:

The page allows users to view and manage their personal information, such as name, username, email, bio, and website links. The page utilizes Riverpod to manage user state and offers editable fields for the name and bio. It also allows users to add or remove website links, with validation for URLs. A profile picture section includes a long-press action that prompts the user to log out. The page uses dialogs for editing fields and confirming actions like adding links or logging out.

Statistic Page:

The page shows key metrics such as the total number of finished quizzes, unfinished quizzes, highest score, and total score. Each statistic is displayed within a Card component for clear visual separation. Additionally, the page includes a reminder section, encouraging the user to start a new quiz.

Guide Page:

The page provides an overview of the app's main features, with descriptions and screenshots for each page. To support this, an assets folder was created to store the images, and the `pubspec.yaml` file was updated to include the image paths under the flutter section.

BottomNavBar widget:

This navigation bar allows seamless navigation between four key sections of the app: the Statistic Page, Profile Page, Quiz List, and Guide Page.

Server-side:

This server side supports two types of quizzes:

Task Generator: These quizzes are created on the fly using specific algorithms (task generators) for different types of logical tasks, such as:

Conditional Statements (via `handleGenerateConditionalTask`), De Morgan's Law (via `handleGenerateDeMorganTask`), and Truth Tables (`handleGenerateTruthTableTask`) Each time these endpoints are called, a specific number of questions is generated, and the number of questions can be easily adjusted in the backend.

Fixed Quizzes: These are pre-defined quizzes stored in a database. The backend retrieves these quizzes based on their difficulty level (easy, medium, or hard) using the `handleGetTask` endpoint. Additionally, new fixed questions can be added with `handleAddTask` and deleted with `handleDeleteTask`.

User Handler :

- **In `handleSignup`**, POST requests allow user registration. The function decodes user details from the request body, generates a UUID, and registers the user using `registerUser()`. If successful, it returns a 200 OK status.
- **`handleLogin`** manages user authentication. It takes login credentials from the request, checks them with `authenticateUser()`, and returns the user data (excluding the password) if valid. On failure, it sends a 401 Unauthorized error.
- **`handleUpdateUserState`** updates user state via POST requests. It verifies credentials with Basic Authentication and, once validated, updates the user's state, saving the changes to the database with `saveUserToDB()`.
- **`handleGetUserState`** retrieves the current user state using Basic Authentication. After verifying credentials, it fetches the user data from the database and returns it as a JSON response.
- **`handleApiPage`**: Provides an API list page for admin users to manage all available APIs. It uses HTTP Basic Authentication to ensure that only authorized admin users can access the page. Upon successful authentication, it serves the `api.html` page.
- **`handleAdminPage`**: Provides an admin dashboard page where admin users can manage quizzes and user data. Similar to `handleApiPage`, it uses HTTP Basic Authentication to verify the user's credentials. Once authenticated, it serves the `admin.html` page to allow admin users to manage the application.
- Finally, **`validateCredentials()`** compares the provided password with the hashed password in the database.

Database :

1. Users Table:

- The users table stores user information such as uuid, name, username, email, and password. The password is securely hashed before being stored in the database.
- The table also includes optional fields like websiteUrls, bio, and description.
- Quiz-related fields like totalQuizNumber track the total number of quizzes a user has completed, and quizScores stores the scores of different quizzes in a formatted string (e.g., "quiz1:5,quiz2:4").

2. Quiz Table:

- The quiz table stores quiz questions, including the question text, a comma-separated string of multiple-choice options, the correct_answer, and the difficulty level (easy, medium, or hard)
- Admin can update or delete tasks from the server side provided HTML page.

Setting up the project

Install Flutter SDK:

1. Download and install the Flutter SDK from the official website. Follow the installation instructions for your operating system (Windows, macOS, or Linux).
2. After installation, verify the setup by running the “flutter doctor” following command in the terminal. This will check if all necessary dependencies for Flutter are installed.
3. Install Android Studio or VS Code, and ensure you have created an Android Virtual Device (AVD) for testing by going to the AVD Manager in Android Studio.
4. Clone the project repository,

```
`git clone https://theogit.fmi.uni-stuttgart.de/fapra\_ss2024/fp\_ss24\_caowo`
```

```
`cd coursework`
```

5. first start server side and default listen on local port 7999.

```
`cd serverside`
```

```
go run .
```

6. then go back to front end page, running the project

```
`cd ../frontend/`
```

```
`flutter devices`
```

```
`flutter run -d <device-id>`
```

7. Make sure the connections between the Flutter frontend, GoLang backend are set up correctly. By default, 'localhost:7999' is used, and due to the emulator issue, the server side api will be convert to "<http://10.0.2.2:7999/>"
8. Once all services are up and running, you should be able to view and interact with the Flutter frontend on the emulator. The front-end can communicate with the GoLang backend, allowing for full interaction with the app's features.
9. If you have an admin account, you can access the backend management interface, navigate to the following URL in your browser: "<http://localhost:7999/admin/>". This interface allows admin to update tasks in the backend and the URL "<http://localhost:7999/admin/api/>" allows admin to monitor API endpoints while interacting with the front-end in real time.

Brief documentation

Architectural

The entire project was developed in 3 parts, "Frontend", "Serverside", "Task Generator"

Frontend:

State Management:

State in the Flutter app is managed using flutter_riverpod, a powerful and flexible package. This was chosen because it simplifies state management across multiple widgets, ensuring that user sessions and quiz data are handled efficiently. It allows for better scalability and cleaner code organization.

Routing:

Navigation is implemented using the go_router package, which provides an easy way to manage page transitions. It allows for smooth navigation between different app sections like the quiz list, profile, and results pages. go_router also supports deep linking, making it versatile for routing needs.

API Communication:

The app communicates with the back-end using the http package, sending HTTP requests to fetch quizzes, manage user sessions, and retrieve results. Error handling is implemented to display appropriate feedback when requests fail, ensuring a smoother user experience.

Serverside:

Database:

The database is structured using SQLite, with two main tables: users for storing user information and authentication details, and quiz for quiz questions. The users table includes fields like uuid, username, and password, while the quiz table stores the question text, options, correct answers, and difficulty levels.

API Structure:

The API is built using Go's net/http package, with RESTful endpoints for key operations such as user authentication, retrieving quizzes, and updating user profiles. These endpoints ensure a clear separation of concerns and efficient data handling between the front-end and back-end.

Authentication:

Secure user authentication is implemented by hashing passwords using the golang.org/x/crypto/bcrypt package. Basic Authentication is used to verify user credentials during login and state updates, ensuring secure access to user data and services.

Task Generator:

In this project, I implemented a system to automatically generate quiz questions for three different types of logic problems, including De Morgan's Law, conditional statements, and truth tables.

Predefined Expression Templates:

1. Each logic generator starts with a set of predefined templates that represent different logical expressions for the specific problem type.
2. Templates include various logical operations (e.g., AND, OR, NOT) and are designed to be flexible enough for random variable insertion.

Random Variable and Expression Generation:

1. Random variables (e.g., p, q, r) are inserted into the predefined templates to create unique logical expressions.
2. For each question, a new random expression is generated to ensure variety across the quizzes.

Correct Answer Calculation:

After generating the random logical expression, the system simplifies it using the appropriate logic rules. The simplified expression becomes the correct answer for the generated question.

Map-Based Unique Answer Handling:

1. The correct answer is stored in a map to ensure uniqueness, preventing duplicate correct answers.

2. This map structure guarantees that only one answer is correct.

Generating Incorrect Answers:

1. Random additional expressions are generated to produce incorrect answers.
2. These wrong answers are also stored in the map, but with the assurance that none duplicate the correct answer.

Ensuring Question Integrity:

1. Using the map, the generator automatically guarantees that the correct answer is unique and that the other three answers are plausible yet incorrect.
2. This process ensures that each question is properly structured with one correct answer and three distinct incorrect ones.

Scalability:

This approach is scalable, allowing the system to generate a large number of unique questions for each logic type, providing diversity and making the quizzes both challenging and comprehensive.

Tests

Tests were not included in the submitted code. However, throughout the development process, certain functions were created to test the interactions between the API and the SQL database. In instances where bugs appeared, I utilized debugging techniques like adding `fmt.Printf` or `print` statements to trace and resolve the issues effectively.

Packages

In this Flutter project, the following key packages are used.

Front end:

`flutter_riverpod (^2.5.1)`: A state management solution that simplifies handling and sharing application state across widgets.

`go_router (^14.2.1)`: A routing package for managing navigation and deep linking within the app.

flutterntoast (^8.2.6): A utility for displaying toast notifications, providing quick feedback to users.

http (^1.2.2): A package for making HTTP requests, facilitating communication with a backend or APIs.

cupertino_icons (^1.0.6): A set of iOS-style icons for providing a native look on Apple devices.

Sever Side:

github.com/google/uuid (v1.6.0): Used for generating unique identifiers (UUIDs) for various purposes, such as user identification.

github.com/mattn/go-sqlite3 (v1.14.22): A driver for SQLite databases, enabling interaction with the SQLite database used in the project.

golang.org/x/crypto (v0.25.0): Provides cryptographic utilities, such as hashing and password encryption (e.g., bcrypt), to secure user credentials and other sensitive data.

Sources

flutter_riverpod (version ^2.5.1)

Source: https://pub.dev/packages/flutter_riverpod

go_router (version ^14.2.1)

Source: https://pub.dev/packages/go_router

flutterntoast (version ^8.2.6)

Source: <https://pub.dev/packages/flutterntoast>

http (version ^1.2.2)

Source: <https://pub.dev/packages/http>

cupertino_icons (version ^1.0.6)

Source: https://pub.dev/packages/cupertino_icons

github.com/google/uuid (version v1.6.0)

Source: <https://github.com/google/uuid>

github.com/mattn/go-sqlite3 (version v1.14.22)

Source: <https://github.com/mattn/go-sqlite3>

golang.org/x/crypto (version v0.25.0)

Source: <https://pkg.go.dev/golang.org/x/crypto>

Flutter Navigation with go_router: go() vs push()

Source: <https://codewithandrea.com/articles/flutter-navigation-gorouter-go-vs-push/>

How to use SQLite with Go

Source: <https://www.allhandsontech.com/programming/golang/how-to-use-sqlite-with-go/>

Flutter SocketException: Connection Refused Error

Source:

<https://stackoverflow.com/questions/60001536/flutter-socketexception-socketexception-os-error-connection-refused-errno>

How to Point to localhost:8000 with the Dart HTTP Package in Flutter

Source:

<https://stackoverflow.com/questions/47372568/how-to-point-to-localhost8000-with-the-dart-http-package-in-flutter>

Photo Slideshow WebApp on GitHub

Source: <https://github.com/askcloudarchitech/photo-slideshow-webapp>

ChatGPT by OpenAI

Source: ChatGPT assisted with structuring and rewriting various sections of the report, including project setup instructions, overview, package explanations, and other clarifications. Used September 2024.

Conclusion

In conclusion, this project provided a valuable learning experience in developing a complete front-end and back-end system. Through building both aspects of the application, I gained a deeper understanding of how to integrate Flutter for the front-end and Go for the back-end. One of the key takeaways was learning how to implement secure user authentication between the front-end and back-end, ensuring that data transmission and user management are handled properly. Additionally, I improved my knowledge of database management, particularly in how to store, retrieve, and manipulate data using SQLite in Go. Overall, this project significantly enhanced my programming skills and gave me a well-rounded experience in full-stack development.